Full Stack Development with MERN

Project Documentation

Date:	30 September 2026
Team ID:	[SWTID-2026-3230]
Project Name:	Auto Ticket Classification using Flow Designer
Maximum Marks:	20 Marks

---

1. Introduction

• Project Title: Auto Ticket Classification using Flow Designer [cite: 37]

• Team Members: TEAM-07 (Lead Developer, ML Engineer, Full Stack Developer, QA Engineer) [cite: 37]

2. Project Overview

• Purpose: The school IT helpdesk receives multiple incident requests daily from students and teachers, such as Wi-Fi issues, projector failures, password problems, and slow computers. Currently, IT staff manually reviews each request and assigns a category, which is time-consuming and inefficient. This project automates ticket classification using ServiceNow Flow Designer by analyzing keywords in the incident's short description and description fields [cite: 37].

• Features: Automated school incident ingestion, keyword-based categorization (Wi-Fi, Projector, Password, Performance), ServiceNow Flow Designer workflow automation, and MERN-based tracking dashboard [cite: 37].

3. Architecture

• Frontend: Developed using React.js with component-based architecture, Redux for state management, and Tailwind CSS for responsive styling [cite: 37].

• Backend: Built with Node.js and Express.js RESTful API architecture handling middleware, authentication, and service controllers [cite: 37].

• Database: MongoDB NoSQL database utilizing Mongoose schemas for storing school incident logs, user profiles, and classification metadata [cite: 37].

4. Setup Instructions

• Prerequisites: Node.js (v18+), MongoDB Atlas / Local Instance, npm package manager [cite: 37].

• Installation: Step-by-step guide to clone the repository, install dependencies via 'npm install' in both client and server directories, and configure .env environment variables (PORT, MONGO_URI, JWT_SECRET) [cite: 37].

5. Folder Structure

• Client: React app structure containing /src/components, /src/pages, /src/redux, and /src/services [cite: 37].

• Server: Node.js backend structure containing /models, /routes, /controllers, and /middleware [cite: 37].

6. Running the Application

• Provide commands to start the frontend and backend servers locally [cite: 37].

o Frontend: npm start in the client directory [cite: 37].

o Backend: npm start in the server directory [cite: 37].

7. API Documentation

• Document all endpoints exposed by the backend: POST /api/tickets/create, GET /api/tickets/classify, GET /api/analytics/metrics [cite: 37].

• Include request methods, parameters, and example responses for school incident routing [cite: 37].

8. Authentication

• Explain how authentication and authorization are handled in the project using JSON Web Tokens (JWT) and bcrypt password hashing [cite: 37].

• Include details about tokens, sessions, and role-based access control for students, teachers, and IT administrators [cite: 37].

9. User Interface

• Provide screenshots or GIFs showcasing different UI features including real-time school ticket tracking graphs and automated flow management panels [cite: 37].

10. Testing

• Describe the testing strategy and tools used including Jest, Supertest, React Testing Library, and UAT validation scripts [cite: 37].

11. Screenshots or Demo

• Provide screenshots or a link to a demo deployed on the staging server to showcase the application.

Demo link : **https://drive.google.com/file/d/1atGFe8-aX1xszmoPoqbm7_ZyGCJvlDbm/view?usp=sharing**

Screen Shot:
https://drive.google.com/drive/folders/1r9mRm_xdQ2NAIv1eLTGyfgK7XBYYMRnj?usp=sharing

12. Known Issues

• Document any known bugs or issues, such as minor dashboard refresh latency during high-volume bulk incident uploads from school users (resolved via optimized scheduled refresh intervals) [cite: 37].

13. Future Enhancements

• Outline potential future features or improvements, such as integrating advanced Generative AI models for automated auto-reply generation to students/teachers and predictive incident resolution time forecasting [cite: 37].